Technical Design Document (TDD)

Project Name: BonPlan.ai
Document Status: Approved for Implementation
Product Vision: Tell us When. We Tell the How.

1.0 System Architecture & Data Flow

This section defines how the core components interact during a complex action, such as a user dragging an event to change its duration.

- **Client Layer (React):** Handles optimistic UI updates. When a drag action occurs, it immediately updates the visual calendar and dispatches an async `PATCH` request to the backend.
- **API Layer (FastAPI):** Receives the request, queries the database for surrounding events, and mathematically calculates the "Ripple Effect" on downstream timestamps.
- **Database Layer (PostgreSQL):** Enforces strict data integrity. It checks the `is_anchor` boolean of downstream events. If the Ripple Effect calculation hits a `true` anchor, the database transaction is aborted.
- **AI Layer (LangChain):** If a gap needs to be filled or an event swapped, FastAPI hands the whitespace parameters to LangChain, which executes the ReAct loop using external tools (Google Maps, Places).
- **External API Layer:** Google Cloud services return real-world transit times and open hours, which LangChain formats into JSON and passes back to FastAPI for database insertion.

2.0 Database Schema (Entity-Relationship Model)

Table: `Users`

Column	Type	Description
id	UUID (PK)	Unique user identifier.
email	String	User email for authentication.
password_hash	String(Nullable)	User's hashed password
auth_provider	String	Is user using "Google" or "Local"
preferences	JSONB	Stores budget, dietary, and pacing preferences.

Table: Trips

Column	Type	Description
id	UUID (PK)	Unique trip identifier.

user_id	UUID (FK)	References Users.id.
source	String	Source city or region.
destination	String	Target city or region.
start_date	Date	First day of the trip.
end_date	Date	Last day of the trip.
pacing	Number	Number 1 to 10 for trip pace.

Table: Days

Column	Type	Description
id	UUID (PK)	Unique day identifier.
trip_id	UUID (FK)	References Trips.id.
date	Date	The specific calendar date.

Table: Events (The Core Engine)

Column	Type	Description
id	UUID (PK)	Unique event identifier.
day_id	UUID (FK)	References Days.id.
title	String	Name of the activity or flight.
type	String	Type of event - Flight, dinner, etc.
description	String	Description of the event.
start_time	Timestamp	ISO 8601 start time.
end_time	Timestamp	ISO 8601 end time.
is_anchor	Boolean	**CRITICAL:** If true, AI cannot modify or delete.
place_id	String	Google Places ID for exact routing.
coordinates	Point	Lat/Lng for distance calculations.

3.0 API Contracts (REST Specifications)

3.1 Initialize Trip

- **Endpoint:** `POST /api/v1/trips`
- **Request Body:** `{ “source”: “London”, "destination": "Paris", "start_date": "2026-05-10", "end_date": "2026-05-15", "pacing": 4 }`
- **Response:** Returns the created Trip ID and initialized empty `Days` array.

3.2 Add Smart Anchor

- **Endpoint:** `POST /api/v1/events`
- **Request Body:** `{ "day_id": "uuid", "title": "Flight Landing", "start_time": "...", "end_time": "...", "is_anchor": true, "place_id": "ChI..." }`
- **Response:** `201 Created` with the saved event object.

3.3 Trigger AI Generation

- **Endpoint:** `POST /api/v1/trips/{trip_id}/generate`
- **Action:** Instructs the backend to calculate whitespace around anchors and trigger LangChain.
- **Response:** Streams SSE (Server-Sent Events) with status updates, followed by the final JSON array of generated `Events`.

3.4 Specific Regeneration (Swap)

- **Endpoint:** `POST /api/v1/events/{event_id}/swap`
- **Request Body:** `{ "context": "Change to an indoor museum." }`
- **Action:** Deletes the target event, maintains the exact time boundary, and queries the LLM for a replacement.
- **Response:** Returns the single new `Event` object to be patched into the React UI.

3.5 Update Event (Drag-and-Drop)

- **Endpoint:** `PATCH /api/v1/events/{event_id}`
- **Request Body:** `{ "new_start_time": "...", "new_end_time": "..." }`
- **Response:** If successful, returns `200 OK` with an array of all subsequent events that were modified by the Ripple Effect. If an anchor is hit, returns `409 Conflict` with the collision details.

4.0 UI/UX State Flow (Optimistic UI Strategy)

Agentic loops inherently introduce latency. To prevent the application from feeling sluggish, the frontend must aggressively manage user perception.

- **Drag-and-Drop Action:** When a user extends a 1-hour block to 2 hours, React immediately renders the change and visually shifts the subsequent events down the calendar.
- **Network State:** The `PATCH` request fires in the background. A subtle, non-blocking loading spinner appears on the modified blocks.
- **Resolution (Success):** The backend confirms the math. The spinner disappears.
- **Resolution (Collision):** If the backend returns a `409 Conflict` (e.g., the Ripple Effect hit an 8:00 PM dinner anchor), React immediately snaps the blocks back to their original positions and renders a toast notification: *"Cannot extend activity; conflicts with locked Smart Anchor."*

5.0 Evaluation & Testing Strategy

Standard unit testing will cover the CRUD operations, but the LLM behavior requires mathematical benchmarking.

- **The Golden Dataset:** A static set of 10 highly complex trip profiles (e.g., arriving at 4:00 PM, departing next morning at 6:00 AM, strict dietary needs).
- **Metric 1: Anchor Violation Rate.** Evaluated programmatically. The test suite fails instantly if the LLM output alters or overlaps any predefined anchor. Target: 0%.
- **Metric 2: Geographic Feasibility.** A separate script verifies that the transit time between AI-generated Event A and Event B is less than the scheduled gap.
- **Metric 3: Token Efficiency.** Logs are parsed to ensure no generation loop exceeds the 5-iteration limit.